

Министерство образования Республики Беларусь

**Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»**

**ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И
УПРАВЛЕНИЯ**

Кафедра интеллектуальных информационных технологий

Отчет

По дисциплине: Аппаратное обеспечение интеллектуальных систем

Выполнил: Евик А.Н., гр. 421702

Проверил: Захаров В.В.

Минск 2026

Лабораторная работа № 2

Представление числовой информации и выполнение в ЭВМ арифметических операций над числами, представленными в разных формах и кодах

Вариант 10

Заданная функция: $\overline{(\overline{x_1} + \overline{x_2}) \cdot \overline{x_2} \cdot x_3}$

Цель: повторение и закрепление материала по преобразованию логических функций, освоение навыков по использованию свойств логических функций, законов и следствий алгебры логики для преобразования логических функций представленных в разных формах.

Алгоритм:

1. Построение таблицы истинности:

Программа позволяет построить результат таблицы истинности, используя стороннюю библиотеку:

Таблица истинности				
x1	x2	x3	f(x)	Вес
0	0	0	1	128
0	0	1	0	64
0	1	0	1	32
0	1	1	1	16
1	0	0	1	8
1	0	1	0	4
1	1	0	1	2
1	1	1	1	1

Числовым индексом называется сумма весов разрядов, соответствующих значению '1'.

Числовой индекс для заданного выражения:

$$i = 187$$

2. Построение совершенной дизъюнктивной нормальной формы.

Программа позволяет строить СДНФ по таблице истинности и выводить ее в аналитической и числовой формах:

```
(СДНФ):  
Аналитическая: (!x1 * !x2 * !x3) v (!x1 * x2 * !x3) v (!x1 * x2 * x3) v (x1 * !x2 * !x3) v (x1 * x2 * !x3) v (x1 * x2 * x3)  
Числовая форма: v(0, 2, 3, 4, 6, 7)
```

3. Построение совершенной конъюнктивной нормальной формы.

Программа позволяет строить СКНФ по таблице истинности и выводить ее в аналитической и числовой формах:

```
(СКНФ) :  
Аналитическая: (x1 + x2 + !x3) ∧ (!x1 + x2 + !x3)  
Числовая форма: ∧(1, 5)
```

Вывод: СКНФ - способ записи логической функции, представляющий собой конъюнкцию элементарных дизъюнкций.

СДНФ - способ записи логической функции, представляющий собой дизъюнкцию элементарных конъюнкций.

Конституента единицы - элементарная конъюнкция, являющееся функцией всех аргументов заданного набора.

Конституента нуля - элементарная дизъюнкция, являющаяся функцией всех аргументов заданного набора.

Числовая форма записи функции является наиболее компактной: вместо развернутых булевых выражений используются лишь наборы номеров конституент. Преимущество данной формы - краткость и простота восстановления как таблицы истинности, так и аналитической записи. Недостатком является снижение наглядности: из записи непосредственно не видна структура функции.

Произвольная форма записи функции является наиболее гибкой: она может сочетать операции отрицания, конъюнкции и дизъюнкции в любом сочетании, включая групповые и общие отрицания. Преимуществом является возможность компактной и интуитивно понятной записи сложных выражений. Главный недостаток — невозможность непосредственного сравнения двух функций без предварительного приведения к нормальной форме.

Листинг кода:

```
import itertools
```

```
class LogicalFunctionSolver:  
    def __init__(self):
```

```

        self.variables = ['x1', 'x2', 'x3']
        self.n_vars = len(self.variables)
        self.truth_table = []
        self.index = 0

    def define_function(self, x1, x2, x3):
        inner = ((not x1) or (not x2)) and (not x2) and x3
        return not inner

    def generate_truth_table(self):
        self.truth_table = []
        print("-" * 30)
        print("Таблица истинности")
        print(f"{'x1':<3} {'x2':<3} {'x3':<3} | {'f(x)':<5} | {'Bec':<5}")
        print("-" * 30)

        binary_values = []
        for j, inputs in enumerate(itertools.product([0, 1],
repeat=self.n_vars)):
            x1, x2, x3 = inputs
            result = int(self.define_function(x1, x2, x3))
            weight = 2 ** (7 - j)

            self.truth_table.append({
                'j': j,
                'inputs': inputs,
                'result': result,
                'weight': weight
            })

            binary_values.append(str(result))
            print(f"{'x1':<3} {'x2':<3} {'x3':<3} | {'result':<5} | {'weight':<5}")

        binary_string = "".join(binary_values)
        self.index = int(binary_string, 2)

        print("-" * 30)
        print(f"Индекс функции i = {self.index}")
        print("-" * 30)

    def get_literal(self, var_name, value):
        if value == 1:
            return var_name
        else:
            return f"!{var_name}"

    def build_sdnf(self):

```

```

        terms = []
        indices = []
        for row in self.truth_table:
            if row['result'] == 1:
                indices.append(row['j'])
                term = " * ".join([self.get_literal(v, val)
                                     for v, val in zip(self.variables, row['inputs'])])
                terms.append(f"({term})")

        sdnf_str = " v ".join(terms) if terms else "0"
        numeric_form = f"v({' ', '.join(map(str, indices))})"
        if indices else "None"
        return sdnf_str, numeric_form, indices

    def build_sknf(self):
        terms = []
        indices = []
        for row in self.truth_table:
            if row['result'] == 0:
                indices.append(row['j'])
                term = " + ".join([self.get_literal(v, 1 -
                                                         val) for v, val in zip(self.variables, row['inputs'])])
                terms.append(f"({term})")

        sknf_str = " ^ ".join(terms) if terms else "1"
        numeric_form = f"^({' ', '.join(map(str, indices))})"
        if indices else "None"
        return sknf_str, numeric_form, indices

    def run(self):
        print("Функция: f = !((!x1 + !x2) * !x2 * x3)")

        self.generate_truth_table()

        sdnf_expr, sdnf_num, sdnf_indices = self.build_sdnf()
        sknf_expr, sknf_num, sknf_indices = self.build_sknf()

        print("\nРЕЗУЛЬТАТЫ ПРЕОБРАЗОВАНИЯ:")
        print("=" * 30)

        print("\n1. (СДНФ):")
        print(f"    Аналитическая: {sdnf_expr}")
        print(f"    Числовая форма: {sdnf_num}")

        print("\n2. (СКНФ):")
        print(f"    Аналитическая: {sknf_expr}")
        print(f"    Числовая форма: {sknf_num}")

```

```

        print("\n3. Числовой индекс функции:")
        print(f"    i = {self.index}")

        print("\n4. Запись в числовой форме (по методу
Мак-Класки / индексная форма):")
        print(f"    f_сднф = {sdnf_num}")
        print(f"    f_скнф = {sknf_num}")
        print("=" * 30)

if __name__ == "__main__":
    solver = LogicalFunctionSolver()
    solver.run()

```